Report: Garbage Collector Tuning Harness for DaCapo-Chopin

hy446 – Managed Runtime Systems **csd4845 – Dimitrios Asimakopoulos**

Introduction

This project measures and compares the performance of three different Java garbage collectors (ParallelGC, G1, and ZGC) across a suite of DaCapo-Chopin benchmarks. For each collector I ran multiple configurations, varying heap size, pause-time goals, and numbers of GC and concurrent threads and harvested key metrics (total run time, GC time, pause statistics, event counts, CPU usage, memory footprint) into a CSV. Under the hood, I use simple shell wrappers to launch each benchmark with the desired -XX: flags, record the output, and parse out the metrics. A set of Python tools then aggregates these raw runs into per-configuration summaries, computes derived values like a “lower-bound overhead” (estimated best-case no-GC runtime), and visualizes trade-offs via Pareto-frontier and bar-chart scripts. What I want to showcase is (1) how GC tuning decisions affect both throughput and pause behavior, (2) how close each collector comes to its theoretical zero-GC limit, and (3) which configurations strike the best balance for different application needs.

I undertook this project because I wanted to see how simple JVM flags can dramatically change performance outcomes, and to pinpoint the true “sweet spots” for each garbage collector. I chose the state-of-the-art DaCapo-Chopin suite to test real-world Java workloads and along the way I sharpened my scripting and automation skills (using shell scripts and regular expressions to parse logs). By digging into those GC logs, I gained a deeper understanding of each collector’s architecture and I built an evaluation plan that follows statistical and reporting rules so my conclusions are both accurate and reproducible.

Flags

In the table below are listed some of the most significant flags that I researched and used in this project along with a short explanation on what they control and which garbage collectors they affect. These flags tune significant parameters in garbage collectors and provide us with customization and adaptability in our configuration. This way, and in combination with the versatility that DaCapo-Chopin benchmarks offer, we can understand deeply the mechanisms behind their operation.

-Xmx/-Xms	Controls the total memory available. Larger heaps can reduce frequency of GC but may increase pause durations.	**all**
-XX:MaxGCPauseMillis	Hints G1's pause-time goal. Lower values drive more concurrent work (higher CPU) to hit sub-N ms pauses. Higher values allow more throughput.	**G1**
-XX:ParallelGCThreads	Number of threads mainly for young collections. More threads can shorten GC pause times at cost of CPU.	**Parallel, G1, ZGC**
-XX:ConcGCThreads	Threads for concurrent phases (mark, cleanup). More threads speed concurrent work, reducing "stop-the-world" time spent waiting on concurrent tasks.	**G1, ZGC**
-XX:MaxTenuringThreshold	How many times an object can survive minor GC before promotion to old gen. Higher values keep objects longer in Survivor space.	**Parallel, G1**
-XX:+AlwaysPreTouch	Forces RAM pages to be faulted in at startup ("pre-touched"), smoothing out later page-fault latencies at cost of longer startup.	**all**
-XX:+UseLargePages	Enables huge pages to reduce TLB misses and kernel overhead, improving latency predictability.	**all (if os supports)**
-XX:SurvivorRatio	Ratio between Eden and each Survivor space. Affects promotion frequency and tenuring.	**Parallel, G1**
-XX:+UnlockExperimental VMOptions -XX:+UseShenandoahGC	Turns on Shenandoah's ultra-low pause collector. Great for sub-millisecond pauses in large heaps. use when G1/ZGC pauses prove too long.	**Shenandoah**

Methodology

- Choose garbage collectors

I selected three JVM GCs to cover the spectrum of design goals. I used ParallelGC as my throughput-optimized baseline, G1 as the modern default that balances pause goals with overall speed via concurrent marking and regionized heaps, and ZGC as the ultra-low-latency, concurrent collector built for large heaps with sub-millisecond pauses.

- Select benchmarks

I ran all 22 DaCapo-Chopin workloads to exercise diverse real-world Java code: web and middleware (Tomcat, Tradebeans, Spring), compilers (Eclipse, PMD), graphics engines (Batik, Sunflow, jMonkeyEngine), databases (Cassandra, H2, Kafka), machine-learning libraries (H2O, BioJava), and more.

- Define the workload matrix

I created a configs.yaml that enumerates the key JVM flag combinations for each GC—varying heap sizes, pause-time targets, and thread counts in a Cartesian product to cover all meaningful permutations.

```yaml
- name: g1-8g-p200-t8-c4
  gc: G1GC
  xmx: 8g
  pause: 200
  pgc: 8
  cgc: 4
  survivorRatio: 8
  maxTenuringThreshold: 15
  pretouch: false
  largePages: false
```

- Automate the execution harness

I created **run_benchmarks.sh** to loop over every <benchmark, config> pair, launch dacapo-chopin.jar for ten iterations each, capture standard output to output.txt and GC traces to gc.log, and ran everything on OpenJDK-21, Ubuntu 20.04 with 16 GB RAM and an Intel i7-3820 × 8.

- Parse logs and extract metrics

Using regular expressions, I scanned each gc.log for pause statistics (min, median, p99, max), total GC time, GC event counts, CPU usage and memory data, and I pulled total runtime from output.txt. I debugged, reran all cases three times, then saved all metrics into results/metrics.csv and wrote a Python script to group by configuration and compute averages. For example some of the regular expressions used appear below.

In the output.tx file we have: ===== DaCapo eclipse PASSED in 12345 msec =====
```
pass_re = re.compile(r'^===== DaCapo.* PASSED in (\d+) msec =====$')
```
Matches the line in the output log that indicates the benchmark finished successfully and extracts the total run time in milliseconds.

In zgc gc.log files we have: Collector: Garbage Collection Cycle 1 / 123,45
```
zgc_re = re.compile(r'Collector: Garbage Collection Cycle\s+[0-9,]+\s*/\s*([0-9,]+)')
```
Matches ZGC log lines that report a GC pause and extracts the pause duration.

Pause Young (G1 Evacuation Pause) (0.123s) 12,345ms
```
gc_time_re = re.compile(r'])]\s*([0-9]+,[0-9]+)ms')
```
Matches GC log lines for non-ZGC collectors to extract the pause time in milliseconds.

The metrics that I extract from these logs are the most useful ones to give us insight on the performance of each configuration. Analytically:

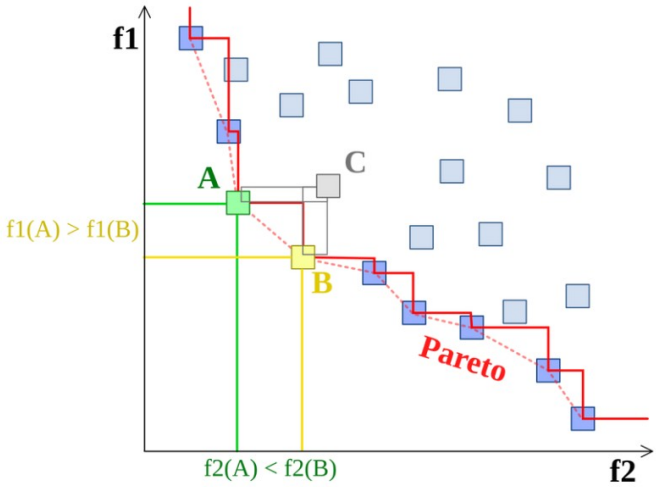
- **total_time_ms**
  I record the end-to-end runtime of the benchmark to measure overall throughput and see how GC tuning impacts total application speed.
- **total_gc_ms**
  I capture the cumulative time spent in all GC work to quantify how much of the total runtime is devoted to garbage collection.
- **pct_gc_time**
  This is total_gc_ms / total_time_ms × 100; it normalizes GC cost so I can compare collectors or configs regardless of absolute runtime.
- **min_pause_ms**
  I note the shortest stop-the-world pause to understand the best-case latency impact under that configuration.
- **p50_ms**
  The median pause time tells me the “typical” GC latency that applications will experience most of the time.
- **p99_ms**
  High-percentile pause shows the worst-of-the-usual GC hiccups—critical for latency-sensitive services that can’t risk long stalls.

- **max_pause_ms**
  I track the single longest pause to catch any extreme outliers that degrade user experience.
- **young_gc_count**
  Counting how often minor collections occur helps me gauge how aggressively the collector is reclaiming short-lived objects.
- **full_gc_count**
  Recording major/full collections indicates how often the collector falls back to a costly, full-heap stop-the-world pass.
- **cpu_time_s**
  Measuring process-level CPU time separates time spent computing from time spent waiting, revealing if GC threads are monopolizing cores.
- **mem_footprint_mb**
  I log peak memory usage to understand each configuration's heap efficiency and its impact on system resource consumption. Basically, RSS tells you how much real RAM your process is using right now.

```
benchmark,config,total_time_ms,total_gc_ms,pct_gc_time,min_pause_ms,p50_ms,p99_ms,max_pause_ms,young_gc_count,full_gc_count, cpu_time_s, mem_footprint_mb
avrora,parallel-4g-t4,11601.0,120.5,1.04,0.3,6.3,14.6,14.6,10,10,193.6,218.2
avrora,parallel-4g-t8,11530.0,102.5,0.89,0.3,5.8,13.4,13.4,10,10,194.4,235.4
avrora,parallel-8g-t4,11302.0,130.1,1.15,0.4,6.1,14.0,14.0,10,10,193.2,297.7
avrora,parallel-8g-t8,11496.0,117.5,1.02,0.3,6.1,14.6,14.6,10,10,193.5,289.0
avrora,g1-4g-p50-t4-c2,11597.0,174.5,1.50,16.5,17.0,20.4,20.4,0,10,195.9,250.5
avrora,g1-4g-p50-t4-c4,11470.0,182.1,1.59,16.3,17.9,20.9,20.9,0,10,193.8,243.4
avrora,g1-4g-p50-t8-c2,11635.0,158.9,1.37,14.4,15.5,18.2,18.2,0,10,196.8,255.6
avrora,g1-4g-p50-t8-c4,11273.0,159.9,1.42,15.3,15.9,17.1,17.1,0,10,190.1,252.2
avrora,g1-4g-p200-t4-c2,11559.0,177.2,1.53,16.2,17.3,19.7,19.7,0,10,194.1,247.3
avrora,g1-4g-p200-t4-c4,11528.0,176.8,1.53,15.8,17.3,20.9,20.9,0,10,199.0,247.5
```

- Perform quantitative analysis

I derived additional metrics (GC overhead percentage) then generated plots of pause distributions and overhead comparisons per GC and heap size. I identified the Pareto-optimal configurations and traced out the Pareto frontier to highlight the best trade-offs.



A point is Pareto-optimal if you cannot improve one metric (e.g. runtime) without worsening the other (e.g. pause time). The Pareto frontier is the set of all such non-dominated points. It traces out the best trade-offs you can achieve.

G1 Results

heap	pause	parallel	concurrent	throughput (ms)	overhead (%)	total gc time (ms)	p99 pause times	gc events	cpu time (s)	memory (mb)
4	50	4	2	7050.35	33.839	1711.18	78.175	58.35	304.29	2420.975
4	50	4	4	7021.55	33.744	1687.66	76.68	57.6	306.25	2437.41
4	50	8	2	6996.7	25.807	1241.49	62.58	50.15	307.91	2452.7
4	50	8	4	7017.15	26.072	1248.78	64.035	50.45	304.59	2434.28
4	200	4	2	6992.75	30.117	1409.94	84.5	40.95	303.595	2436.23
4	200	4	4	7028.45	30.576	1406.85	80	40.95	303.91	2441.845
4	200	8	2	6987.75	24.389	1091.21	64.42	40	304.84	2441.605
4	200	8	4	6996.65	24.477	1090.83	66.021	39.75	303.585	2447.75
8	50	4	2	7004.1	36.055	1752.6	93.896	46.65	305.72	3711.34
8	50	4	4	6999.05	34.692	1713.83	94.005	45.1	304.56	3692.655
8	50	8	2	7019.5	27.378	1274.40	76.9	37.6	307.05	3865.325
8	50	8	4	7010.8	26.712	1245.40	79.05	36.2	307.46	3787.35
8	200	4	2	6993.9	29.686	1272.6	97.896	26.55	302.025	3894.24
8	200	4	4	7017.25	29.362	1257.99	98.25	26.35	304.455	3890.115
8	200	8	2	7229.211	24.127	1010.64	81.458	25	304.8	3878.41
8	200	8	4	7001.85	24.286	997.53	79.705	25.1	305.26	3888.82

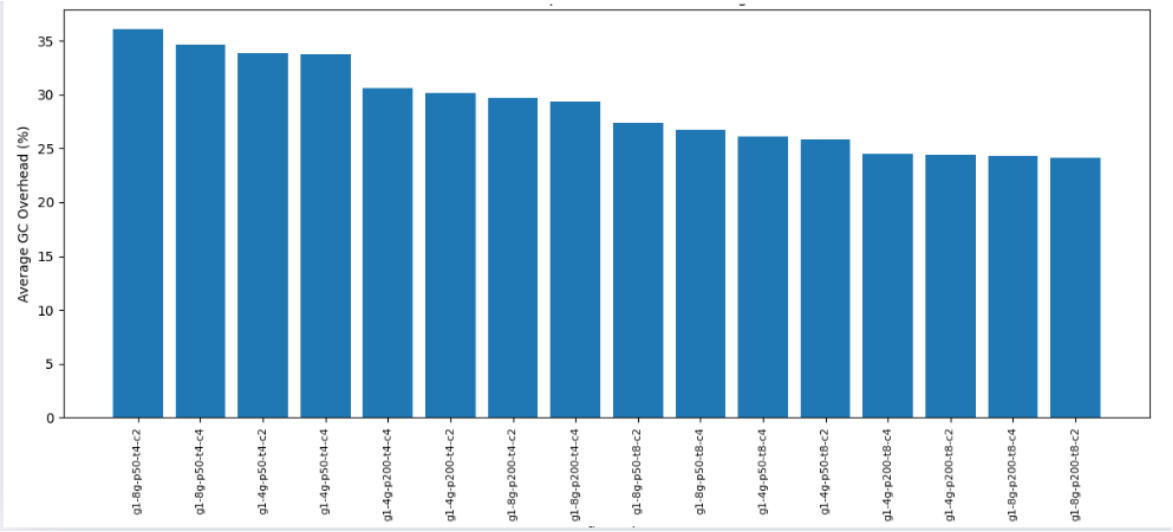
The detailed results for the G1 configurations are shown in the table above. The best performing configurations for each metric are shown in green and the worst ones in red. The conclusions that we can make from these data and for this particular garbage collector are:

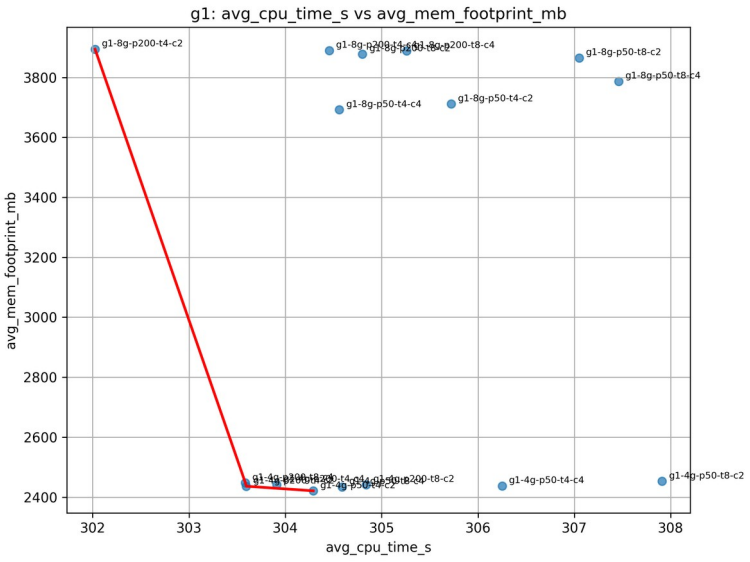
- More parallel threads consistently deliver better throughput and lower GC overhead.
- A tighter pause goal (50 ms) reduces high-percentile pauses noticeably, at only a small cost in overall runtime.
- Increasing the heap to 8 GB does not improve throughput in this test range—it actually slows some runs despite mildly lowering GC%.
- Varying the number of concurrent threads (c2 vs c4) has only a minor effect on both latency and speed.
- In the bar chart, the 16 G1 configs naturally fall into four performance clusters:

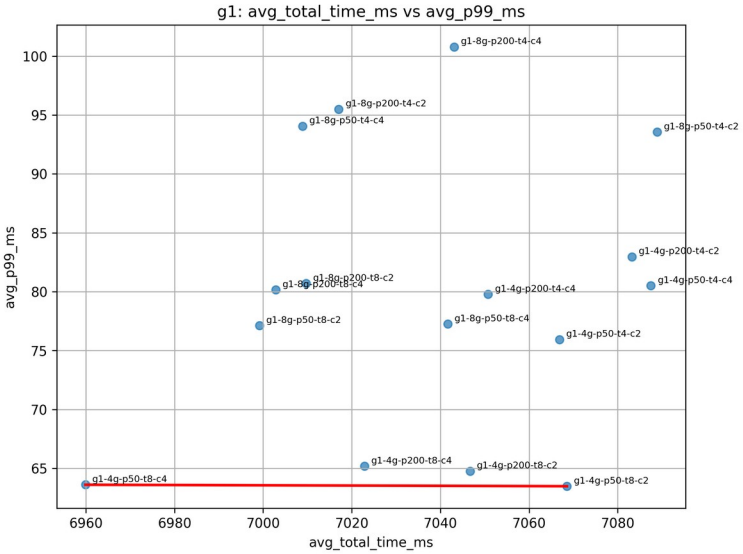
  Worst: 50 ms pause with 4 parallel threads
  Next: 200 ms pause with 8 parallel threads
  Next: 50 ms pause with 8 parallel threads
  Best: 200 ms pause with 8 parallel threads

  This grouping shows that the combination of a relaxed pause-time goal and high parallelism yields the strongest throughput, while strict pause goals with low parallelism suffer most

- 4 GB heap configurations perform better under the 50 ms pause goal—tighter control pays off on a smaller heap.
- 8 GB heap configurations do best with the 200 ms pause goal—the extra memory reduces pressure, so a looser goal unlocks higher throughput.
- CPU usage is remarkably stable across all G1 configurations, varying by only a few percent.
- Tighter pause goals sometimes edge CPU time slightly upward, reflecting extra work to meet latency constraints.
- Increasing parallel or concurrent thread counts has minimal impact on overall CPU consumption.
- Doubling the heap from 4 GB to 8 GB raises the memory footprint by roughly half again, with only marginal CPU-time benefits.
- Pause-time and thread adjustments tune latency without significantly altering compute or memory costs.





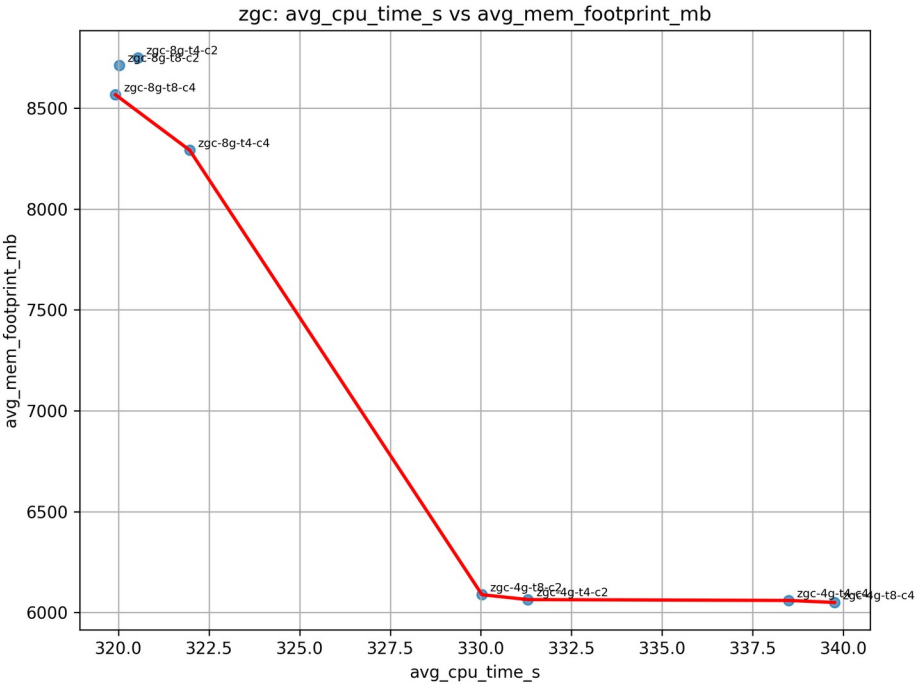


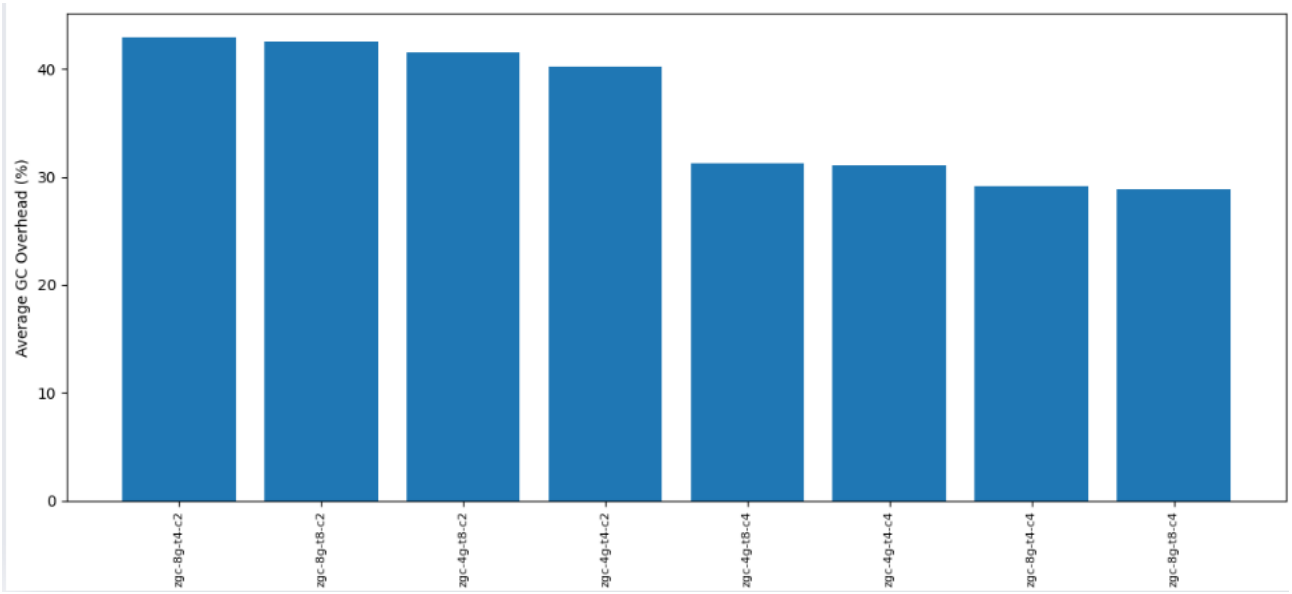
ZGC Results

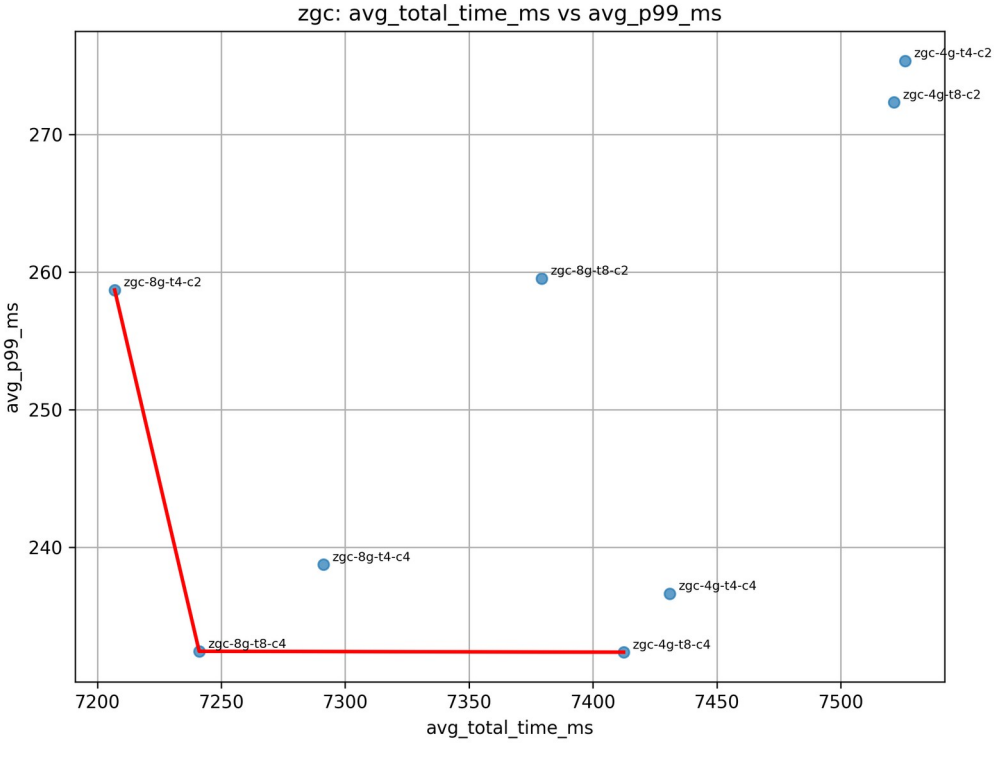
heap size	parallel threads	concurrent threads	throughput (ms)	overhead (%)	total gc time (ms)	p99 pause times	gc events	cpu time (s)	memory footprint (mb)
4	4	2	7471.65	40.223	3243.255	257.01	10.5	331.3	6064.14
4	4	4	7352.15	31.042	2323.73	232.995	10.5	338.49	6059.925
4	8	2	7436.2	41.505	3260.065	271.49	10.5	330.03	6088.42
4	8	4	7362.05	31.243	2368.985	226.2	10.5	339.755	6049.905
8	4	2	7183.4	42.983	3063.09	271.75	10.5	320.54	8749.43
8	4	4	7197.95	29.116	2055.075	235.575	10.5	321.965	8291.29
8	8	2	7150	42.52	3000.47	258.995	10.5	320.02	8712.54
8	8	4	7176.2	28.89	2036.7	241.66	10.5	319.92	8565.985

Similar to the G1, here we can see the results of the ZGC configurations in detail. We notice that:

- Across both heap sizes and parallel-thread counts, bumping ZGC's concurrent workers from 2 to 4 drops GC overhead and reduces p99 pauses, matching ZGC's design to shift more work into concurrent phases. This can easily be visualized in the bar graph where the best performing configurations are those with 4 concurrent threads.
- Switching parallel GC threads from 4 to 8 barely moves throughput or overhead, reflecting ZGC's emphasis on concurrent rather than parallel stop-the-world work.
- Growing from 4 GB to 8 GB gives a modest drop in overhead and total GC time—consistent with ZGC's regionized heap reducing frequency of remapping work.
- Memory usage scales with heap size, confirming ZGC's resident set tracks the configured heap.
- Smaller heaps drive up overall CPU consumption compared to larger heaps.
- Adding more concurrent threads only slightly shifts CPU load, with effects varying by heap size.
- ZGC relies heavily on CPU cycles relative to wall-clock time to achieve its low-pause design.


zgc: avg_cpu_time_s vs avg_mem_footprint_mb
avg_mem_footprint_mb
avg_cpu_time_s
zgc-8g-t4-c2
zgc-8g-t8-c2
zgc-8g-t8-c4
zgc-8g-t4-c4
zgc-4g-t8-c2
zgc-4g-t4-c2
zgc-4g-t4-c4
zgc-4g-t8-c4


Average GC Overhead (%)
zgc-8g-t4-c2
zgc-8g-t8-c2
zgc-4g-t8-c2
zgc-4g-t4-c2
zgc-4g-t8-c4
zgc-4g-t4-c4
zgc-8g-t4-c4
zgc-8g-t8-c4

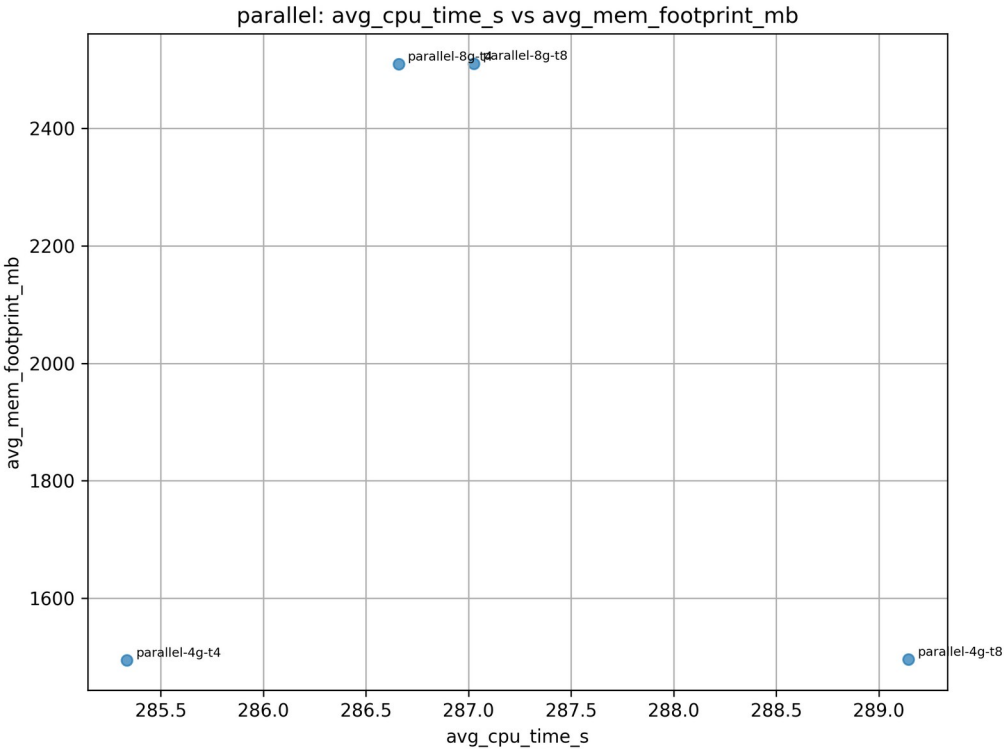

zgc: avg_total_time_ms vs avg_p99_ms
avg_p99_ms
avg_total_time_ms
zgc-4g-t4-c2
zgc-4g-t8-c2
zgc-8g-t8-c2
zgc-8g-t4-c2
zgc-8g-t4-c4
zgc-4g-t4-c4
zgc-8g-t8-c4
zgc-4g-t8-c4

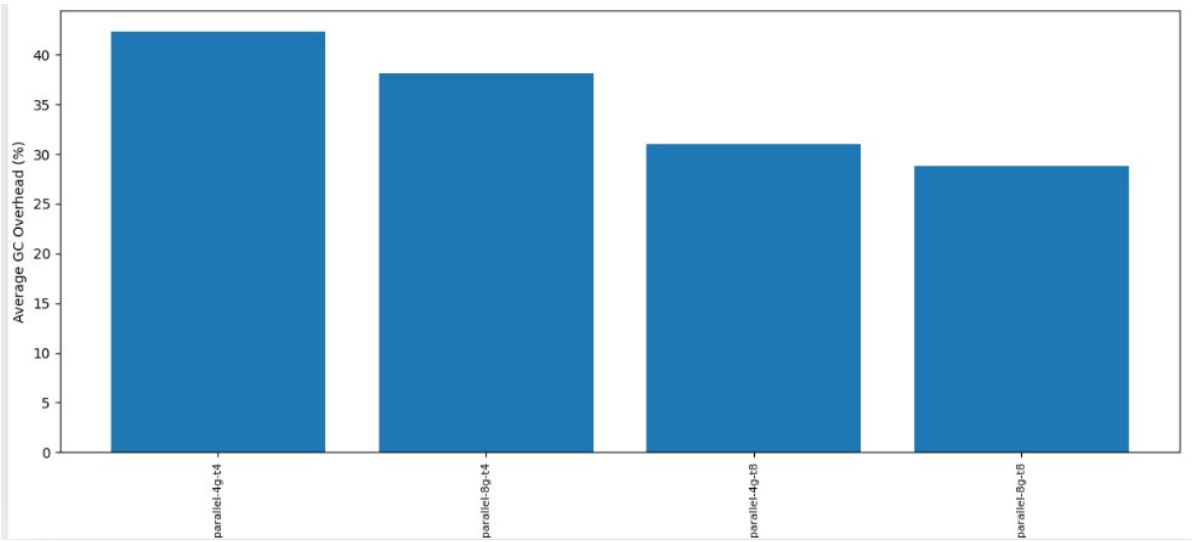
ParallelGC Results

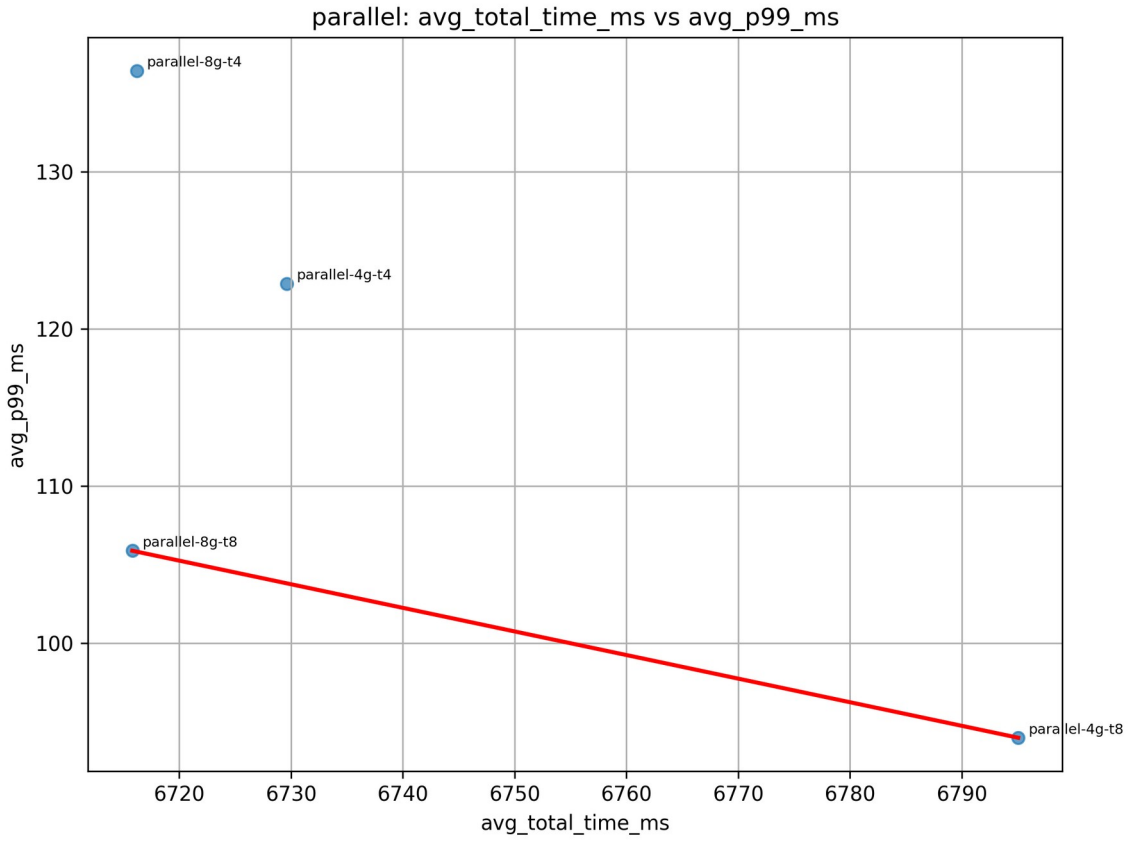
heap size	parallel threads	throughput (ms)	overhead (%)	total gc time (ms)	p99 pause times	gc events	cpu time (s)	memory (mb)
4	4	6735.85	42.3	1663.665	114.57	75	285.335	1494.33
4	8	6719.6	30.987	1231.68	92.24	74.3	289.145	1495.475
8	4	6721.95	38.113	1519.265	135.67	46.5	286.66	2509.49
8	8	6756.75	28.858	1178.54	103.845	46.6	287.025	2510.455

For the ParallelGC we make these conclusion from the metrics above: Adding parallel GC threads improves both speed and pause times

- Moving from 4 to 8 threads on a 4 GB heap cuts average runtime, lowers GC overhead and reduces p99 pauses, exactly as the ParallelGC design predicts: more collector threads shorten both collection duration and overall stall time.
- Larger heap reduces GC frequency and overhead, but can lengthen individual pauses. Doubling from 4 GB to 8 GB with 4 threads drops GC overhead from 42.3 % to 38.1 % and cuts event count from 75 to 46.5, showing fewer collections consistent with the fact that each stop-the-world scan must touch a bigger heap.
- Best latency–throughput trade-off comes from high threads on moderate heap. The fastest, lowest-pause run is 4 GB + 8 threads. This matches theory: ample collector threads on a right-sized heap give maximum throughput with minimal pause.
- Changing the number of GC threads has virtually no effect on application CPU time.
- Doubling the heap size roughly doubles the memory footprint while leaving CPU time nearly unchanged.
- GC configuration tweaks shift memory usage far more than they shift compute cost.
- Keeping a smaller heap delivers similar compute performance at a significantly lower memory cost.


parallel: avg_cpu_time_s vs avg_mem_footprint_mb
parallel-8g-t4
parallel-8g-t8
parallel-4g-t4
parallel-4g-t8
avg_mem_footprint_mb
avg_cpu_time_s
2400
2200
2000
1800
1600
285.5
286.0
286.5
287.0
287.5
288.0
288.5
289.0


Average GC Overhead (%)
40
35
30
25
20
15
10
5
0
parallel-4g-t4
parallel-8g-t4
parallel-4g-t8
parallel-8g-t8


parallel: avg_total_time_ms vs avg_p99_ms
parallel-8g-t4
parallel-4g-t4
parallel-8g-t8
parallel-4g-t8
avg_p99_ms
avg_total_time_ms
130
120
110
100
6720
6730
6740
6750
6760
6770
6780
6790

Benchmark Observations

By analyzing the original metrics.csv file which categorizes each configuration by benchmark I made some interesting observations.

- In H2, zgc-4g-t4-c2 spends 42 050.6 ms in GC, showing ZGC can be inefficient when allocation rates or heap sizing overwhelm its concurrent model.
- For Avrora (zgc-4g-t4-c2), min_pause_ms is 0.0, matching ZGC's ultra-low-latency design. In FOP, this gives ZGC a clear edge in latency-sensitive tasks.
- ZGC excels on FOP but performs poorly on Tradebeans, highlighting that its benefits depend on each application's memory behavior and access patterns.
- PMD runs with ParallelGC show high full_gc_count and p99 pauses around 82 ms, underscoring ParallelGC's weakness with fragmented heaps.
- Lusearch sees a young_gc_count of 173 but few full GCs, indicating efficient young-gen reclamation but potential old-gen churn and many small pauses.
- In BioJava, g1-4g-p50-t4-c2 vs. g1-4g-p200-t4-c2 show almost identical runtimes, suggesting that tighter pause targets don't always change performance for some workloads.
- ZGC configurations (e.g., zgc-8g) consistently show extreme memory usage (up to 24.5 GB in H2 and 13.5 GB in Tradesoap), often 3-10x higher than Parallel/G1, with no proportional CPU time improvement.
- For Batik, ZGC (zgc-4g-t8-c4) reduces CPU time to 16.13s (vs. 121-144s for G1/Parallel) but consumes 2.1 GB memory, highlighting a CPU-memory trade-off.
- H2's ZGC configurations use 24+ GB memory and 1,200+ CPU seconds, far exceeding Parallel's 3.6 GB / 746s, indicating ZGC's unsuitability for memory-heavy workloads.
- For benchmarks like Avrora and FOP, Parallel GC maintains low CPU time (~193s and ~47s) with moderate memory (~200-800 MB), showing efficiency for lighter workloads.
- ZGC-8g configurations in Biojava spike memory to 10.8+ GB (vs. 2.8 GB for Parallel) while increasing CPU time (121-133s vs. 110-116s), suggesting ZGC adds overhead without gains.
- G1 often balances CPU and memory (e.g., Tomcat: 455s CPU / 2.8 GB vs. ZGC's 447s / 3.8 GB), making it a safer default for mixed workloads.

Lower Bound Overhead

I calculate the lower-bound overhead (LBO) to understand how much of each run's time is truly "unavoidable" work versus GC overhead. To do this, I first compute each run's application -only time as **app_time = avg_total_time_ms – avg_total_gc_ms**, then find the smallest app_time across all configs (my proxy for a zero-GC run). For any configuration, I express its overhead relative to that ideal minimum by **LBO_overhead_pct = 100 × (avg_total_time_ms – min_app_time) / avg_total_time_ms**. When I apply this to the top performers, I see that the G1 champions (g1-4g-p200-t8-c2 and g1-4g-p50-t8-c2) sit about 21,5% overhead, indicating they're within ~3 pp of their measured GC% and close to the theoretical best. The ParallelGC winners (parallel-4g-t8 and parallel-8g-t8) show even more headroom (18.3 % and 17.5 %), meaning a larger fraction of their runtime is devoted to GC work that could, in principle, be optimized away. ZGC's best (zgc-8g-t8-c4) barely improves over its measured 28.9 %, confirming it already operates near its zero-GC limit.

Best Configurations	Measured GC%	LBO Overhead%
g1-4g-p200-t8-c2	24.39 %	21.47 %
g1-4g-p50-t8-c2	25.81 %	21.57 %
parallel-4g-t8	30.99 %	18.33 %
parallel-8g-t8	28.86 %	17.45 %
zgc-8g-t8-c4	28.89 %	28.40 %